



Object-Oriented Design in Smart Healthcare Systems

Applying OOAD Principles to Modern Healthcare Software

V V NAVEEN KUMAR
192321018
BTECH-IT

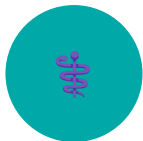
What is a Smart Healthcare System?

A digital platform that connects patients, doctors, appointments, and medical records to deliver efficient, coordinated care.



Patients

Personal info, history,
contacts



Doctors

Specialty, schedule,
credentials



Appointments

Booking, time, status



Medical Records

Diagnosis, treatment,
reports

Class & Object

Class

A blueprint defining attributes and behaviors of an entity.

Object

A concrete instance of a class created at runtime.

```
class Patient {  
    patientId: String  
    name: String  
    age: Integer  
    medicalHistory: List  
    contactInfo: String  
  
    bookAppointment()  
    viewRecords()  
    updateProfile()  
}
```

```
patient1 = new Patient("A123",  
    "Alice", 34)
```

Encapsulation

Bundling data and methods together, and restricting direct access to internal state to protect sensitive information.

Private Data

- SSN
- diagnosis
- medical history

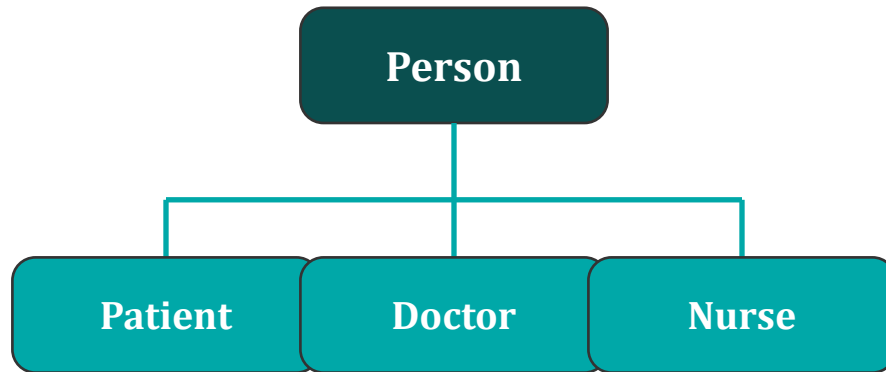
Public Methods

getDiagnosis()
updateRecord()



Inheritance

Creating new classes from existing ones, reusing common attributes and behaviors while adding specialized features.



Common Attributes

- name
- age
- contact
- address
- id

Polymorphism

The same interface behaves differently depending on the underlying object type.

Common Interface: `Appointment.schedule()`

In-Person

`schedule()`
select room
& time

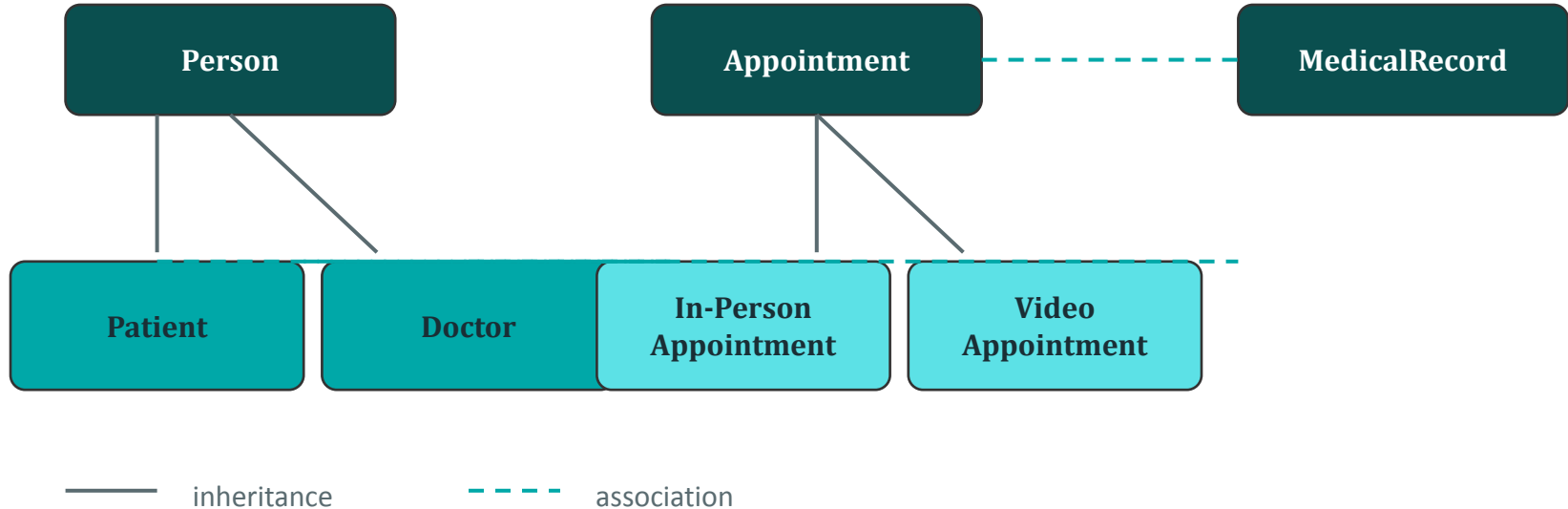
Video Call

`schedule()`
generate
video link

Emergency

`schedule()`
flag as
urgent

Healthcare System Class Architecture



Scalability

M

Modularity

Each class is an independent unit that can be developed, tested, and deployed separately.

R

Reusability

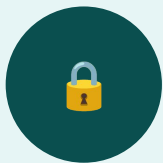
Common behavior is inherited, reducing code duplication across the growing system.

E

Extensibility

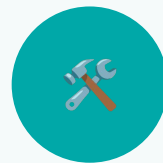
New features (e.g., telemedicine) are added by creating new classes without breaking existing code.

Security & Maintainability



Security

- Private fields hide sensitive data
- Getters/setters enforce access rules
- Role-based permissions protect records



Maintainability

- Self-contained classes are easier to debug
- Inheritance centralizes shared logic
- Polymorphism simplifies testing
- Changes in one class rarely break others

Key Takeaways

Class & Object

Model real-world healthcare entities as reusable blueprints and instances.

Encapsulation

Protect patient data by hiding internals and exposing only safe interfaces.

Inheritance

Reuse common attributes across doctors, patients, and staff.

Polymorphism

Handle diverse appointment types and workflows through a shared interface.

Object-oriented design transforms smart healthcare systems into secure, scalable, and maintainable software.